

TRANSLITLAB

v1.1.0 · Plugin Documentation

Transliteration Lab for the CompLing-AI Engine

BhoomiTech Heritage and Development Foundation

1. Overview

TranslitLab is a browser-based test and configuration module for the CompLing-AI transliteration engine, delivered as a WordPress plugin. It provides a single-page lab interface accessible via a shortcode, giving authorised users the ability to test, tune, and manage every aspect of the Roman ↔ BPM (Bengali Phonetic Mapping) transliteration pipeline — without touching code or the server directly.

1.1 What TranslitLab does

- **Tests both engines side-by-side:** run any input text through the JavaScript (browser) engine and the PHP (server) engine simultaneously and compare their output and debug logs.
- **Manages the lexicon:** view, search, add, edit, delete, and reorder forward (Roman → BPM) and reverse (BPM → Roman) map entries organised by section.
- **Edits processing rules (Snippets):** create, enable, disable, reorder, import, and export pre/loop/post processing rules (regex or literal replacements) per direction and hook stage.
- **Configures engine settings:** adjust virama, ZWJ/ZWNJ characters, inherent vowel, non-linking consonants, and section ordering independently for each engine.
- **Shows debug logs:** inspect step-by-step processing traces from the last run for each engine.
- **Resets to defaults:** restore maps, snippets, and settings to the bundled defaults at any time.
- **Controls access:** restrict lab access to a dedicated "Transliterator" WordPress role with per-user approval and per-user (or global) tab-visibility settings.

1.2 Plugin at a glance

Attribute	Value
Plugin name	TransLit-Lab
Version	1.1.0
Author	BhoomiTech Heritage and Development Foundation
Requires WordPress	6.0 or later
Tested up to	WordPress 6.7
Requires PHP	8.0 or later
License	GPLv2 or later
Text domain	translitlab
Frontend shortcode	[translitlab]
Admin menu location	Settings → TranslitLab

2. Architecture

The plugin is structured as a thin WordPress integration layer over a self-contained PHP transliteration engine, mirrored in JavaScript for the browser. All persistent data lives in four custom database tables inside the WordPress database.

2.1 File structure

```
translitlab/
├── translitlab.php           ← Plugin bootstrap (constants, hooks, loader)
├── uninstall.php           ← Hard-delete routine (tables, options, role)
├── includes/
│   ├── class-engine.php     ← PHP transliteration engine (TranslitEngine)
│   ├── class-db.php         ← Database layer (TranslitLab_DB)
│   ├── class-ajax.php       ← WordPress AJAX action handlers (TranslitLab_Ajax)
│   ├── class-frontend.php   ← Shortcode + asset enqueue (TranslitLab_Frontend)
│   ├── class-admin.php      ← Settings page (TranslitLab_Admin)
│   └── class-role.php       ← Custom role + access logic (TranslitLab_Role)
├── assets/
│   ├── engine.js           ← JavaScript transliteration engine (mirrors PHP)
│   ├── app.js              ← Frontend SPA (tabs, AJAX calls, UI logic)
│   └── style.css            ← Lab UI stylesheet
└── data/
    ├── forwardMap.json      ← Default Roman → BPM lexicon
    ├── reverseMap.json      ← Default BPM → Roman lexicon
    ├── defaults.json        ← Default snippets
    ├── js-engine-settings.default.json ← Default JS engine settings
    └── php-engine-settings.default.json ← Default PHP engine settings
```

2.2 Classes

Class	File	Responsibility
TranslitEngine	class-engine.php	Pure PHP engine: forward & reverse transliteration, snippet hooks, virama/ZWJ logic
TranslitLab_DB	class-db.php	All reads and writes to the four plugin tables; seeds defaults on activation
TranslitLab_Ajax	class-ajax.php	Nine wp_ajax_ handlers; auth guard wraps every endpoint
TranslitLab_Frontend	class-frontend.php	[translitlab] shortcode; conditional asset enqueue; access gate
TranslitLab_Admin	class-admin.php	Settings → TranslitLab admin page: role, access, and tab-visibility management
TranslitLab_Role	class-role.php	Creates/removes the "transliterator" role; user_can_access() and user_tab_config() helpers

2.3 Database schema

The plugin creates four tables on activation using dbDelta():

wp_translit_forward_map / wp_translit_reverse_map

```
id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY
section     VARCHAR(100)   DEFAULT "general"    -- e.g. consonants, matras
map_key     VARCHAR(100)
char (reverse)
```

```

map_val      VARCHAR(500)                -- BPM char (forward) or Roman
glyph (reverse)
sort_order  INT          DEFAULT 0
UNIQUE KEY section_order (section, sort_order)

```

wp_translit_snippets

```

id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY
snippet_key VARCHAR(100) UNIQUE -- stable machine identifier
label       VARCHAR(255)        -- human label shown in UI
direction   VARCHAR(20)         -- "forward" | "reverse" | "both"
hook_stage   VARCHAR(20)         -- "pre" | "loop" | "post"
sort_order  INT
is_active    TINYINT(1)         -- 1 = enabled, 0 = disabled
source       VARCHAR(100)        -- "builtin" | "custom"
logic_description TEXT           -- human description
js_body      LONGTEXT           -- JSON rule array for JS engine
php_body     LONGTEXT           -- JSON rule array for PHP engine
UNIQUE KEY dir_stage_order (direction, hook_stage, sort_order)

```

wp_translit_settings

```

id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY
engine       VARCHAR(10) -- "js" or "php"
setting_key  VARCHAR(100)
setting_val  LONGTEXT    -- scalar value or JSON-encoded array
UNIQUE KEY engine_key (engine, setting_key)

```

3. Installation

3.1 Requirements

Component	Minimum version
WordPress	6.0
PHP	8.0
MySQL / MariaDB	Any version supported by your WordPress installation
Browser	Any modern browser with ES6+ and Fetch API support

3.2 Install steps

1. Download the translitlab-v1_1_0.zip file.
2. **Method A — Upload in the dashboard:** Go to Plugins → Add New → Upload Plugin, select the ZIP file, click Install Now, then Activate.
3. **Method B — FTP / SSH:** Extract the ZIP and upload the translitlab/ folder to /wp-content/plugins/. Activate via Plugins → Installed Plugins.
4. On activation the plugin automatically: registers the "Transliterator" WordPress role, creates the four database tables, and seeds them from the bundled JSON defaults.

5. Open Settings → TranslitLab to grant access to users and configure tab visibility (see Section 5).
6. Create or edit any page/post and add the shortcode [translitlab] where you want the lab to appear.

3.3 What happens on activation

- **Role creation:** TranslitLab_Role::create_role() adds a "transliterators" role with the read capability only. If the role already exists (e.g. after reactivation), it is left untouched.
- **Table installation:** TranslitLab_DB::install() runs dbDelta() for all four tables, then calls maybe_seed().
- **Seeding:** maybe_seed() checks whether wp_translit_snippets is empty. If so, it imports forwardMap.json, reverseMap.json, defaults.json, and both engine-settings JSON files.

3.4 Deactivation vs deletion

Action	Effect
Deactivate plugin	Role hook fires TranslitLab_Role::remove_role(), which deliberately does nothing — users and data are preserved.
Delete plugin	uninstall.php runs: drops all four tables, deletes translitlab_approved_users and translitlab_default_tabs options, removes translitlab_tabs user meta, and calls remove_role("transliterators").

Important

Deleting the plugin is irreversible. Export your maps, snippets, and engine settings from the lab UI before uninstalling if you want to keep them.

4. The Lab Interface

The lab is a single-page application rendered inside the [translitlab] shortcode. It consists of a header with a navigation bar and five tabs. Which tabs are visible depends on each user's access configuration (Section 5).

4.1 Test tab (🔗)

The Test tab lets you run a string through both engines simultaneously.

Direction toggle

Two buttons at the top of the panel switch the direction of transliteration:

- **Forward (Roman → BPM):** input is treated as a Roman-script phonetic string; the engines output Bengali Unicode (BPM).
- **Reverse (BPM → Roman):** input is a Bengali Unicode string; the engines output the Roman phonetic representation.

Input field

A plain-text area. Type or paste the string to transliterate and click Run (or use the keyboard shortcut).

Engine output panels

Panel	Description
JS Engine (frontend / browser)	Runs engine.js entirely in the browser. Appears immediately after clicking Run. Useful for testing the live client-side engine behaviour.
PHP Engine (server / WordPress)	Sends the input to the server via AJAX (translitlab_run_php action), runs class-engine.php, and returns the output. Verifies that the two engines agree.

Each panel shows the transliterated output and, if the Debug tab has been used, a link to view the step log.

Run button

Triggers both engines. The button is labelled ☐ Run. While processing, it shows a spinner. Both engine results update when their respective calls complete.

4.2 Snippets tab (☐)

Snippets are processing rules applied to the text before, during, or after the core map-lookup loop. Each snippet has:

Field	Description
Key (snippet_key)	Stable machine identifier; unique across all snippets. Cannot be changed after creation.
Label	Human-readable name displayed in the UI.
Direction	"forward", "reverse", or "both". Controls which transliteration direction the snippet applies to.
Hook stage	"pre" runs before the loop; "loop" runs inside the character-by-character loop; "post" runs after the loop.
Active toggle	Enable or disable the snippet without deleting it.
Sort order	Within the same direction+stage, snippets are applied in sort-order ascending. Drag-and-drop reordering is supported.
Source	"builtin" for shipped defaults; "custom" for user-added rules.
JS body / PHP body	JSON arrays of replacement rules consumed by the respective engines.

Rule format (JS body / PHP body)

Each body is a JSON array of rule objects. The engines support two rule types:

Simple replacement

```
[
  { "type": "replace", "from": "kh", "to": "ख", "regex": false }
]
```

Regex replacement

```
[
  { "type": "replace", "from": "/([aeiou])r/u", "to": "$1र", "regex": true }
]
```

```
]
```

Loop-stage context filters

Loop-stage rules may include extra fields that restrict when they fire:

Field	Effect
context_output_ends_with	Rule fires only if the current output buffer ends with this string.
context_prev_was_consonant	Rule fires only if (true) or only if not (false) the previous character was a consonant.
context_prev_was_matra	Rule fires only if (true) or only if not (false) the previous character was a matra.
context_next_token_in	Rule fires only if the next token in the remaining input belongs to the named section (e.g., "consonants").

Config overrides (pre stage only)

A pre-stage snippet can supply a "config" key to override engine toggles for this run:

```
{ "config": { "hardcode_consonant_cluster": false } }
```

Snippets tab actions

- **Add:** Creates a new empty snippet row for editing.
- **Save:** Persists the current snippet list to the database via the save_snippets AJAX action.
- **Import ZIP:** Upload a previously exported ZIP containing snippets JSON. Merges with existing snippets.
- **Export ZIP:** Downloads a ZIP of the current snippet list as JSON.
- **Reorder:** Drag snippet rows to reorder them within the same direction+stage group.
- **Delete:** Removes a snippet from the list (takes effect after Save).

4.3 Maps tab (□)

The Maps tab gives a structured view of the forward and reverse lexicon. The map is split into sections:

Section	Contents
consonants	Consonant glyphs and their BPM equivalents.
matras	Dependent vowel signs (diacritics attached to consonants).
independent_vowels	Stand-alone vowel glyphs.
digits	Numeral characters.
punctuation	Punctuation and special typographic characters.
special	Additional or override mappings (processed first by the engine).

Map viewer features

- **Search/filter:** Type in the filter box to narrow visible entries by key or value.
- **Add entry:** Click + Add to append a new key–value pair to a section.
- **Edit entry:** Click on any cell in the table to edit the key or value inline.
- **Delete entry:** Click the delete icon next to any row.

- **Reorder within section:** Drag rows to reorder. Longest-key-first ordering is enforced at runtime by the engine regardless of visual order; dragging changes the `sort_order` column.
- **Switch direction:** Toggle between the Forward and Reverse maps.
- **Save:** Persists changes to the database via the `save_map` AJAX action.
- **Reset to defaults:** Restores the currently selected map to the bundled JSON defaults.

4.4 Engines tab (□)

The Engines tab exposes the low-level settings that control how each engine processes text. Settings can be configured independently for the JS engine and the PHP engine.

Engine settings reference

Setting key	Default	Description
<code>virama</code>	U+09CD (Bengali Halanta)	Unicode virama character inserted between consonants in a cluster.
<code>zwj</code>	U+200D (Zero Width Joiner)	ZWJ inserted to request a ligature between characters.
<code>zwnj</code>	U+200C (Zero Width Non-Joiner)	ZWNJ inserted to suppress ligature or indicate a word boundary.
<code>inherent_vowel</code>	"o"	Roman romanisation of the inherent vowel appended after a bare consonant in reverse mode.
<code>non_linking_consonants</code>	["r", "R", "ng", "M", "NG", "Ng", ":", "H", "^", "~"]	Consonants that never form conjuncts; the virama+ZWJ cluster-linking logic is skipped for these.
<code>hardcode_independent_vowel</code>	true	Whether to apply the built-in independent-vowel detection logic.
<code>hardcode_explicit_zwnj</code>	true	Whether to convert apostrophes to ZWNJ in the built-in logic.
<code>hardcode_consonant_matra</code>	true	Whether to apply the built-in consonant+matra combination logic.
<code>hardcode_consonant_cluster</code>	true	Whether to apply the built-in consonant-cluster + halanta logic.
<code>section_order</code>	["special", "consonants", "matras", "independent_vowels", "digits", "punctuation"]	Order in which sections are flattened into the lookup map. Earlier sections take precedence for overlapping keys.

4.5 Debug tab (□)

The Debug tab shows the step-by-step processing log from the most recent Run. Each log line indicates the rule or built-in logic step that was applied and the position in the input. Log prefixes:

Prefix	Meaning
[FWD IND VOWEL]	Independent vowel matched in forward direction.
[ZWNJ]	Explicit ZWNJ (apostrophe) inserted.

[FWD CONS+MATRA]	Consonant + matra combination matched.
[FWD CLUSTER]	Consonant cluster + halanta (virama) applied.
[MATCH]	A map entry matched and was substituted.
[PASS]	Character passed through unchanged (no map match).
[INHERENT]	Inherent vowel appended after a bare consonant (reverse mode).
[PRE/POST]	A pre- or post-stage snippet rule was applied.
[LOOP]	A loop-stage snippet rule was applied.
[CONFIG]	A snippet overrode an engine config flag.
[ERROR]	An invalid direction or other error occurred.

5. Access Control

5.1 How access works

Three conditions determine whether a logged-in WordPress user can see the lab:

7. **Must be logged in.** Guests always see a "Please log in" notice.
8. **Must have the Transliterator role.** Administrators automatically bypass this check. Non-admin users without the role see a "Permission denied" notice.
9. **Must be in the approved-users list.** Even with the Transliterator role, a user also needs explicit approval in Settings → TranslitLab before they can access the lab.

5.2 Settings → TranslitLab

The admin settings page is located at Settings → TranslitLab and is accessible only to users with the `manage_options` capability (administrators).

Shortcode info card

Displays the `[translitlab]` shortcode and a reminder that non-logged-in visitors and unapproved users see polite placeholder messages.

Default Tab Visibility card

Five checkboxes (one per tab: Test, Snippets, Maps, Engines, Debug) control which tabs are shown to all Transliterators by default. This global setting can be overridden per user.

User Access & Tab Control table

Lists every non-administrator user registered on the site. Columns:

Column	Description
User	Display name and email address.

Current Roles	All existing WordPress roles for this user (excluding the Transliterator role, shown separately).
Transliterator Role	Checkbox to add or remove the Transliterator role. Unchecking automatically unchecks Lab Access.
Lab Access	Checkbox to add or remove the user from the approved-users list. Disabled unless the Transliterator role is checked.
Tab Visibility	"Override defaults" checkbox. When checked, an inline set of five tab checkboxes becomes active, allowing a custom tab set for this user. When unchecked, the global defaults apply and the per-user translitlab_tabs meta is deleted.

A live search box filters the table by display name or email.

Saving

Click Save All Settings to persist global tab defaults, per-user roles, access approval, and tab overrides in a single form submission. A success notice is shown on redirect.

6. AJAX API

All communication between the frontend JavaScript application and the server uses the standard WordPress AJAX endpoint (admin-ajax.php). Every action requires the user to be logged in, to have TranslitLab access, and to supply a valid nonce (translitlab_nonce).

6.1 Authentication

The nonce is generated on page load by `wp_create_nonce("translitlab_nonce")` and passed to the frontend via `wp_localize_script` as `TranslitLabConfig.nonce`. Every request sends it as a nonce field. The server-side guard calls `check_ajax_referer("translitlab_nonce", "nonce")` before any action handler runs.

6.2 Request format

The JavaScript app sends FormData with two fields:

- `action`: the AJAX action name, e.g. `translitlab_load`
- `nonce`: the page-load nonce
- `payload`: a JSON-encoded string containing action-specific parameters

6.3 Action reference

Action	Payload fields	Response
<code>translitlab_load</code>	(none)	{ snippets, forward_map, reverse_map, php_settings, js_settings }
<code>translitlab_run_php</code>	input, direction, snippets?, forward_map?, reverse_map?, php_settings?	{ ok, output, log[] }

translitlab_get_engine_source	(none)	{ js: "...engine.js source...", php: "...class- engine.php source..." }
translitlab_save_snippets	snippets[]	{ ok, corrections }
translitlab_save_map	type ("forward" "reverse"), data{}	{ ok, corrections }
translitlab_save_engine_settings	engine ("js" "php"), data{}	{ ok }
translitlab_reset_defaults	which ("all" "forward" "reverse" "snippets" "php_settings" "js_settings")	{ ok [, data] }
translitlab_reorder_map_section	type, section, ids[]	{ ok }
translitlab_check_sort_order_conflicts	type	{ ok, conflicts[] }

7. The Transliteration Engine

TranslitLab ships identical transliteration logic in two languages — PHP (class-engine.php) and JavaScript (engine.js) — so that server-side rendering and browser-side rendering can be cross-validated.

7.1 Forward transliteration pipeline (Roman → BPM)

The forward() method processes input character-by-character using a stateful loop. At each position it attempts the following steps in order:

10. **Pre-hooks:** Active "pre" snippets in sort_order are applied to the full input string before the loop begins.
11. **Config hooks:** Pre-stage snippets may supply a "config" object to override engine flags for this run.
12. **Map preparation:** The forward map is flattened to a key→value lookup in section_order priority and sorted longest-key first. Consonant, matra, and independent-vowel sub-maps are also extracted and sorted.
13. **Loop — Step 1: Independent vowel detection** (hardcode_independent_vowel). If at a word boundary (position 0 or after a non-alpha character, and the previous consumed text does not end with an independent vowel), the longest matching independent-vowel key is substituted and the position advances.
14. **Loop — Step 2: Explicit ZWNJ** (hardcode_explicit_zwnj). An apostrophe character is replaced by U+200C (ZWNJ) to suppress consonant cluster formation.
15. **Loop — Step 3: Consonant + matra** (hardcode_consonant_matra). Matches a consonant followed immediately by a matra. Special handling for "o" (inherent vowel), "y"/"Y" (with possible second matra), and apostrophe matra (ZWNJ).
16. **Loop — Step 4: Consonant cluster + halanta** (hardcode_consonant_cluster). Matches a bare consonant (no following matra) followed by another consonant. Inserts virama (U+09CD) between them, plus ZWJ for "r" as initial consonant in most cases (reph), to request a conjunct glyph.
17. **Loop — Step 5: Loop snippets** Active "loop" snippets (sorted by sort_order) are tried. Each snippet's rule set is applied to the state object (output buffer, prev flags, current position). The first snippet that modifies the state marks the position as handled and breaks.

18. **Loop — Step 6: Core map lookup** The flattened lookup is searched for the longest key matching at the current position. If found, the value is appended to the output and the position advances. If not found, the character is passed through unchanged.
19. **Post-hooks:** Active "post" snippets are applied to the complete output string after the loop ends.

7.2 Reverse transliteration pipeline (BPM → Roman)

The reverse() method is simpler because BPM is a fully-specified Unicode script — no inherent-vowel ambiguity or cluster-detection rules are needed for parsing. It proceeds as follows:

20. **NFC normalisation** (if the Intl.Normalizer / normalizer_normalize() extension is available).
21. **Pre-hooks** as above.
22. **Loop snippets** are tried first at each position, before map lookup.
23. **Core map lookup** using the reverse map. After matching a consonant, the engine checks the next character: if it is neither a matra nor a virama, the inherent_vowel is appended to the output.
24. **Pass-through** for any character that does not match the map.
25. **Post-hooks** as above.

7.3 Constructor parameters (PHP)

```
new TranslitEngine(  
    array $forward_map,    // Loaded from DB or forwardMap.json  
    array $reverse_map,    // Loaded from DB or reverseMap.json  
    array $settings = []   // Optional; falls back to hardcoded defaults  
);
```

7.4 Public API

```
// PHP  
$engine = new TranslitEngine($fwd, $rev, $settings);  
$result = $engine->transliterate($input, "forward", $snippets);  
// $result => ["output" => "বাংলা", "log" => ["[MATCH] b → ব", ...]]  
  
// JavaScript (engine.js exports a global TranslitEngine or module)  
const engine = new TranslitEngine(fwdMap, revMap, settings);  
const { output, log } = engine.transliterate(input, "forward", snippets);
```

8. Data Flow

8.1 Page load

26. WordPress serves the page containing the [translitlab] shortcode.
27. TranslitLab_Frontend::render_shortcode() checks login and access; if authorised, renders the app HTML and calls enqueue_assets().
28. wp_localize_script() injects TranslitLabConfig (ajaxUrl, nonce, enabledTabs) into the page.

29. On DOMContentLoaded, app.js calls translitlab_load via AJAX to retrieve all snippets, maps, and engine settings into memory.
30. The JS engine is initialised in-browser with the loaded map and settings.

8.2 Run cycle

31. User types input and clicks Run.
32. app.js runs the JS engine synchronously in the browser; result is displayed in the JS panel immediately.
33. app.js sends a `translitlab_run_php` AJAX request with the input, direction, and current in-memory snippets/map/settings.
34. `class-ajax.php::handle_run_php()` instantiates TranslitEngine with the provided data (or falls back to the DB if data not sent).
35. The PHP engine runs and returns { output, log }; the JS displays the result in the PHP panel.

8.3 Save cycle

36. User edits a snippet, map entry, or setting.
37. User clicks Save. app.js sends the relevant action (`save_snippets`, `save_map`, or `save_engine_settings`).
38. The AJAX handler validates and writes to the database via TranslitLab_DB.
39. On success, app.js updates its in-memory state and the JS engine is reinitialised.

9. WordPress Integration Details

9.1 Hooks registered

Hook	Type	Handler
plugins_loaded	action	Boots Admin, Frontend, Ajax classes
admin_menu	action	TranslitLab_Admin::add_menu() — adds Settings → TranslitLab
admin_init	action	TranslitLab_Admin::register_settings()
admin_post_translitlab_save_users	action	TranslitLab_Admin::handle_save_users()
admin_enqueue_scripts	action	TranslitLab_Admin::enqueue_admin_assets()
wp_enqueue_scripts	action	TranslitLab_Frontend::maybe_enqueue()
shortcode: translitlab	shortcode	TranslitLab_Frontend::render_shortcode()
wp_ajax_translitlab_*	action ×9	TranslitLab_Ajax handlers (logged-in users only)

9.2 Options stored

Option name	Type / Description
translitlab_approved_users	Serialised array of int user IDs approved for lab access.

translitlab_default_tabs	Serialised array of tab slugs visible by default (e.g. ["test", "maps"]).
--------------------------	---

9.3 User meta

Meta key	Type / Description
translitlab_tabs	Per-user array of visible tab slugs; overrides the global default when non-empty. Deleted when admin unchecks "Override defaults".

9.4 Assets

Handle	Description
translitlab-style	assets/style.css — lab UI styles, enqueued on pages containing the shortcode
translitlab-engine	assets/engine.js — JS transliteration engine, enqueued in the footer
translitlab-jszip	JSZip 3.10.1 from cdnjs, used for import/export ZIP in Snippets and Maps
translitlab-app	assets/app.js — frontend SPA logic, depends on engine + jszip, enqueued in the footer

10. Frequently Asked Questions

Who can see the lab on the front end?

Logged-out visitors see a "Please log in" message. Logged-in users without the Transliterator role or without explicit lab-access approval see a permission notice. Administrators always have access. Everyone else needs both the Transliterator role and approval via Settings → TranslitLab.

Will the JS and PHP engines always produce the same output?

They are designed to mirror each other and should produce identical output for the same map, settings, and snippets. The Test tab exists precisely to validate this. If they diverge, the debug logs from both engines will show where the processing differs.

Can I add entries to the map without losing my snippets?

Yes. The Maps tab saves only the map; the Snippets tab saves only snippets. Each Save action is scoped to its own data set.

What does Reset to Defaults do?

It re-imports the relevant bundled JSON file(s) into the database. For maps this replaces the entire map for the selected direction. For snippets it replaces all snippets. For engine settings it replaces all settings for the selected engine. Other data sets are not affected unless you select "all".

How do I export my current maps and snippets?

Use the Export ZIP buttons in the Snippets and Maps tabs. The downloaded ZIP contains JSON files that can be re-imported on another installation.

What happens if I delete the plugin?

All four database tables are dropped, the `translitlab_approved_users` and `translitlab_default_tabs` options are removed, `translitlab_tabs` user meta is deleted from all users, and the "transliterator" role is removed from the system. User accounts themselves are not affected.

Can two Transliterator users work on the same installation simultaneously?

Yes. Each user has a separate session and nonce. However, if two users edit the same map section and both click Save, the last write wins. There is no collaborative locking mechanism.

11. Troubleshooting

Symptom	Resolution
Lab shows "Please log in" even when logged in	Check that the [translitlab] page is not cached at the server or CDN level while you are logged in. Clear caches.
Lab shows "Permission denied"	Visit Settings → TranslitLab, enable the Transliterator role for the user, and check Lab Access.
JS engine produces output but PHP engine shows an error	Check the WordPress error log. Ensure PHP 8.0+; confirm the four plugin tables exist; check that AJAX is not blocked by a security plugin.
JS and PHP outputs differ	Open the Debug tab and compare logs from both engines. A differing snippet body (<code>js_body</code> vs <code>php_body</code>) or a missing PHP Intl extension (for NFC normalisation in reverse mode) are common causes.
Import ZIP fails	Ensure the ZIP was exported from a TranslitLab installation and has not been corrupted. The JSON inside must be a valid array.
Snippets do not appear to fire	Confirm the snippet's <code>is_active</code> flag is enabled, the direction matches the current test direction, and the <code>hook_stage</code> is correct (<code>pre/loop/post</code>). Check the Debug tab log for [PRE/POST] or [LOOP] entries.
Database tables missing after activation	Deactivate and reactivate the plugin, which re-runs the <code>install()</code> and seeding routines.

12. Changelog

v1.1.0 — Current release

- Initial public release.
- Dual-engine (JS + PHP) test panel.
- Snippets editor with `pre/loop/post` hook stages.
- Forward and reverse lexicon map viewer with section-level reordering.
- Per-engine settings panel (`virama`, `ZWJ/ZWNJ`, inherent vowel, non-linking consonants, section order, `hardcode` flags).
- Debug log panel.

- Reset-to-defaults for maps, snippets, and engine settings individually or all at once.
- Transliterator custom WordPress role with per-user access approval and tab-visibility control.